

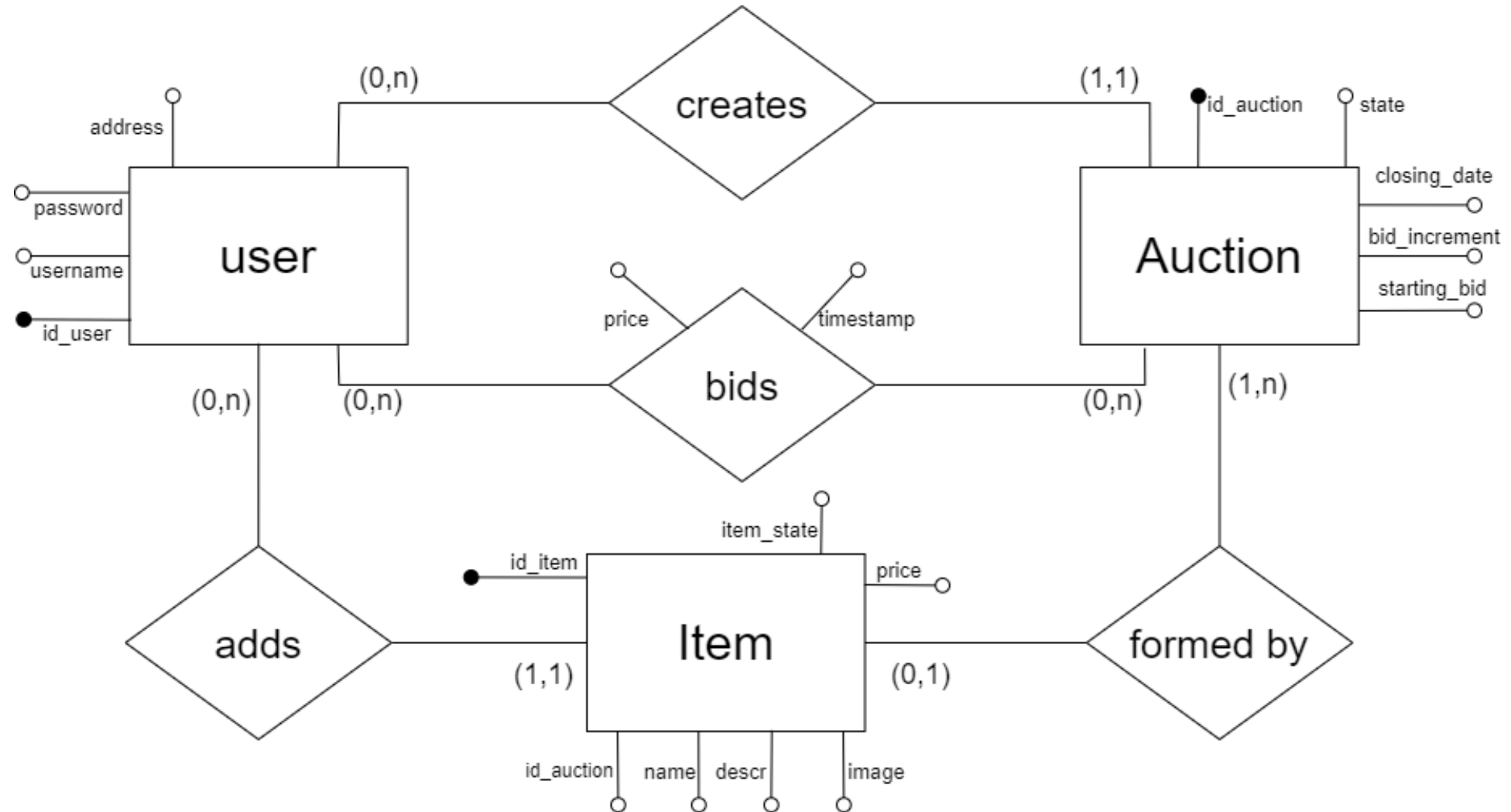
Esercizio 1 – Pure HTML

Alberto Biffi – Silvia Firenze

Esercizio 1: Aste Online (Pure HTML) – Analisi DB

- Un'applicazione web consente la gestione di **aste** online. Gli **utenti** accedono tramite login e possono vendere e acquistare all'asta. La HOME page contiene due link, uno per accedere alla pagina VENDO e uno per accedere alla pagina ACQUISTO. La pagina VENDO mostra una lista delle aste create dall'utente e non ancora chiuse, una lista delle aste da **lui** create e chiuse e due form, una per **creare un nuovo articolo** e una per **creare una nuova asta** per vendere gli articoli dell'utente. Il primo form inserisce nuovi articoli nel database e il secondo mostra l'elenco degli articoli disponibili nel database e dà la possibilità di selezionarne più di uno. Un **articolo** ha **codice**, **nome**, **descrizione**, **immagine** e **prezzo**. Un'asta **comprende uno o più articoli** messi in vendita, il prezzo iniziale dell'insieme di articoli, il rialzo minimo di ogni offerta (espresso come un numero intero di euro) e una scadenza (data e ora, es. 19-04-2021 alle 24:00). Il prezzo iniziale dell'asta è ottenuto come somma del prezzo degli articoli compresi nell'offerta. Lo stesso **articolo non può essere incluso in aste diverse**. Una volta venduto, un **articolo non deve essere più disponibile per l'inserimento in ulteriori aste**. La lista delle aste è ordinata per data+ora crescente. L'elenco riporta: codice e nome degli articoli compresi nell'asta, offerta massima, tempo mancante (numero di giorni e ore) tra il momento (data ora) del login e la data e ora di chiusura dell'asta. Cliccando su un'asta compare una pagina DETTAGLIO ASTA che riporta per un'asta aperta tutti i dati dell'asta e la lista delle offerte (**nome utente**, **prezzo offerto**, **data e ora** dell'offerta) ordinata per data+ora decrescente. Un bottone CHIUDI permette all'utente di chiudere l'asta se è giunta l'ora della scadenza (si ignori il caso di aste scadute ma non chiuse dall'utente e non ci si occupi della chiusura automatica di aste dopo la scadenza). Se l'asta è chiusa, la pagina riporta tutti i dati dell'asta, il nome dell'aggiudicatario, il prezzo finale e l'**indirizzo** (fisso) **di spedizione** dell'utente. La pagina ACQUISTO contiene una form di ricerca per parola chiave. Quando l'acquirente invia una parola chiave la pagina ACQUISTO è aggiornata e mostra un elenco di aste aperte (la cui scadenza è posteriore alla data e ora dell'invio) per cui la parola chiave compare nel nome o nella descrizione di almeno uno degli articoli dell'asta. La lista è ordinata in modo decrescente in base al tempo (numero di giorni e ore) mancante alla chiusura. Cliccando su un'asta aperta compare la pagina OFFERTA che mostra i dati degli articoli, l'elenco delle offerte pervenute in ordine di data+ora decrescente e un campo di input per inserire la propria offerta, che deve essere superiore all'offerta massima corrente di un importo pari almeno al rialzo minimo. Dopo l'invio dell'offerta la pagina OFFERTA mostra l'elenco delle offerte aggiornate. La pagina ACQUISTO contiene anche un elenco delle offerte aggiudicate all'utente con i dati degli articoli e il prezzo finale.
- **Entities**, **attributes**, **relationships**

Database design



Logical scheme

User(id_user, username, password, address);

Auction(id_auction, id_user, starting_bid, bid_increment, auction_state, closing_date);

Item(id_item, id_auction, id_user, price, item_state, image, name, descr);

Bid(id_user, id_auction, price, timestamp);

Foreign keys:

Auction.id_user -> User.id_user

Item.id_user -> User.id_user

Bid.id_user -> User.id_user, Bid.id_auction -> Auction.id_auction

Tables definitions

```
CREATE TABLE `auction` (  
  `id_user` int NOT NULL,  
  `id_auction` int NOT NULL AUTO_INCREMENT,  
  `starting_bid` int DEFAULT NULL,  
  `bid_increment` int DEFAULT NULL,  
  `closing_date` date NOT NULL,  
  `auction_state` enum('opened','closed') DEFAULT NULL,  
  PRIMARY KEY (`id_auction`),  
  KEY `id_user` (`id_user`),  
  CONSTRAINT `auction_ibfk_1` FOREIGN KEY (`id_user`)  
REFERENCES `user` (`id_user`)  
) ENGINE=InnoDB
```

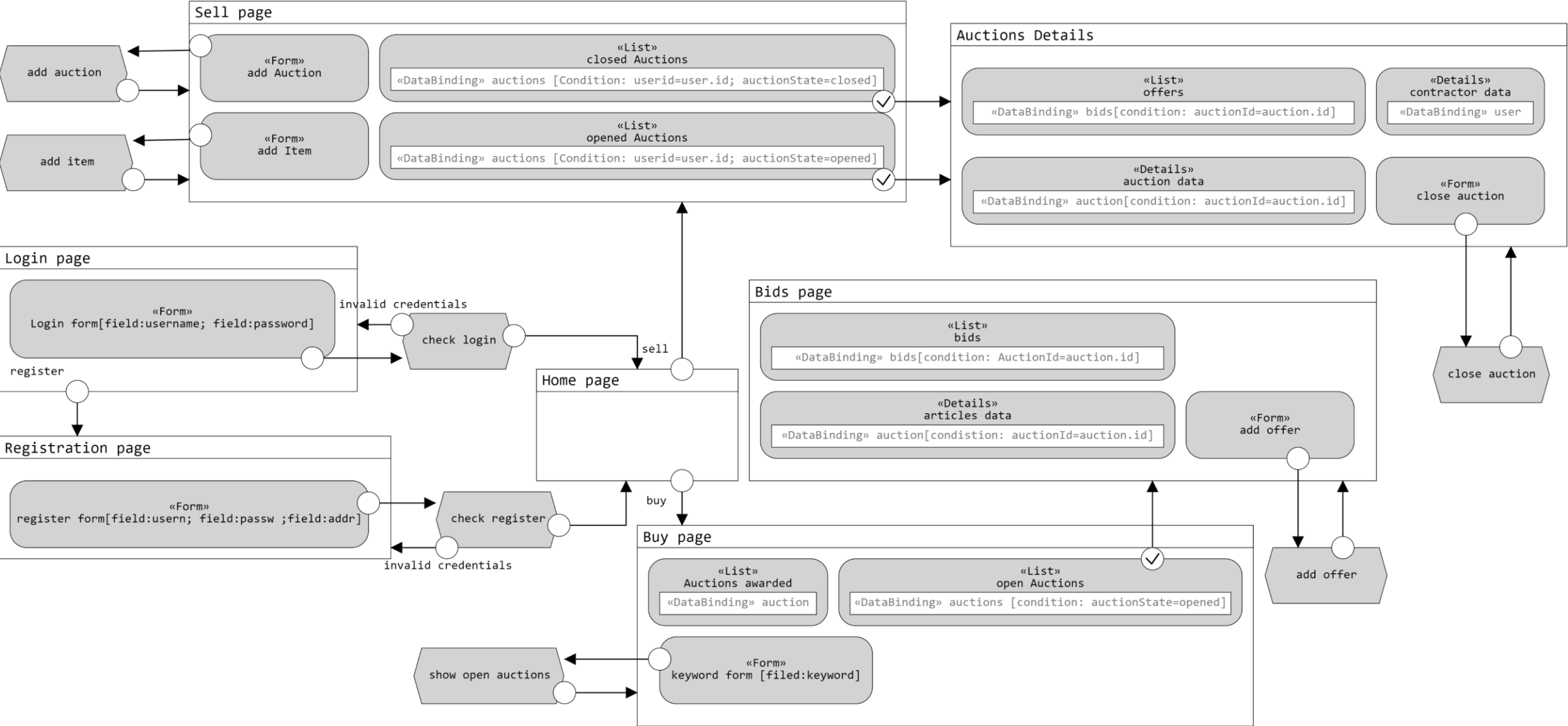
```
CREATE TABLE `bid` (  
  `id_user` int NOT NULL,  
  `id_auction` int NOT NULL,  
  `price` int NOT NULL,  
  `timestamp` datetime NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
  KEY `id_user` (`id_user`),  
  KEY `bid_ibfk_2_idx` (`id_auction`),  
  CONSTRAINT `bid_ibfk_1` FOREIGN KEY (`id_user`)  
REFERENCES `user` (`id_user`),  
  CONSTRAINT `bid_ibfk_2` FOREIGN KEY (`id_auction`)  
REFERENCES `auction` (`id_auction`) ON DELETE RESTRICT ON  
UPDATE RESTRICT  
) ENGINE=InnoDB
```

Tables definitions

```
CREATE TABLE `item` (  
  `id_user` int NOT NULL,  
  `id_item` int NOT NULL AUTO_INCREMENT,  
  `id_auction` int DEFAULT NULL,  
  `price` int NOT NULL,  
  `name` varchar(20) NOT NULL,  
  `description` varchar(255) NOT NULL,  
  `img_name` varchar(100) CHARACTER SET utf8mb3 COLLATE  
utf8mb3_general_ci NOT NULL,  
  `item_state` enum('not_auctioned','auctioned','sold')  
DEFAULT NULL,  
  PRIMARY KEY (`id_item`),  
  KEY `id_user` (`id_user`),  
  CONSTRAINT `item_ibfk_1` FOREIGN KEY (`id_user`)  
REFERENCES `user` (`id_user`)  
) ENGINE=InnoDB
```

```
CREATE TABLE `user` (  
  `id_user` int NOT NULL AUTO_INCREMENT,  
  `username` varchar(20) NOT NULL,  
  `password` varchar(16) NOT NULL,  
  `address` varchar(100) NOT NULL,  
  PRIMARY KEY (`id_user`),  
  UNIQUE KEY `username_UNIQUE` (`username`)  
) ENGINE=InnoDB
```

IFML design



Analisi requisiti applicazione

- Un'applicazione web consente la gestione di aste online. Gli utenti **accedono** tramite **login** e possono vendere e acquistare all'asta. La **HOME page** contiene **due link**, uno per **accedere** alla **pagina VENDO** e uno per **accedere** alla **pagina ACQUISTO**. La pagina VENDO mostra una lista delle aste create dall'utente e non ancora chiuse, una **lista delle aste da lui create e chiuse** e **due form**, una per **creare un nuovo articolo** e una per **creare una nuova asta** per vendere gli articoli dell'utente. Il primo form inserisce nuovi articoli nel database e il secondo **mostra l'elenco degli articoli disponibili nel database e dà la possibilità di selezionarne più di uno**. Un articolo ha codice, nome, descrizione, immagine e prezzo. Un'asta comprende uno o più articoli messi in vendita, il prezzo iniziale dell'insieme di articoli, il rialzo minimo di ogni offerta (espresso come un numero intero di euro) e una scadenza (data e ora, es. 19-04-2021 alle 24:00). Il prezzo iniziale dell'asta è ottenuto come somma del prezzo degli articoli compresi nell'offerta. Lo stesso articolo non può essere incluso in aste diverse. Una volta venduto, un articolo non deve essere più disponibile per l'inserimento in ulteriori aste. La lista delle aste è ordinata per data+ora crescente. L'elenco riporta: codice e nome degli articoli compresi nell'asta, offerta massima, tempo mancante (numero di giorni e ore) tra il momento (data ora) del login e la data e ora di chiusura dell'asta. **Cliccando su un'asta** compare una **pagina DETTAGLIO ASTA** che **riporta** per un'asta aperta **tutti i dati dell'asta** e la **lista delle offerte** (nome utente, prezzo offerto, data e ora dell'offerta) ordinata per data+ora decrescente. **Un bottone CHIUDI** permette all'utente di **chiudere l'asta se è giunta l'ora della scadenza** (si ignori il caso di aste scadute ma non chiuse dall'utente e non ci si occupi della chiusura automatica di aste dopo la scadenza). Se l'asta è chiusa, la pagina riporta **tutti i dati dell'asta, il nome dell'aggiudicatario, il prezzo finale e l'indirizzo (fisso) di spedizione dell'utente**. La **pagina ACQUISTO** contiene una form di ricerca per parola chiave. Quando l'acquirente **invia una parola chiave** la pagina ACQUISTO è **aggiornata e mostra** un elenco di aste aperte (la cui scadenza è posteriore alla data e ora dell'invio) per cui la parola chiave compare nel nome o nella descrizione di almeno uno degli articoli dell'asta. La lista è ordinata in modo decrescente in base al tempo (numero di giorni e ore) mancante alla chiusura. **Cliccando su un'asta aperta** compare la **pagina OFFERTA** che **mostra i dati degli articoli, l'elenco delle offerte** pervenute in ordine di data+ora decrescente e un campo di input per inserire la propria offerta, che deve essere superiore all'offerta massima corrente di un importo pari almeno al rialzo minimo. Dopo l'**invio dell'offerta** la pagina OFFERTA **mostra l'elenco delle offerte aggiornate**. La **pagina ACQUISTO** contiene anche un **elenco delle offerte aggiudicate all'utente con i dati degli articoli e il prezzo finale**.
- **Pages, view components, events, actions**

Completamento specifiche

- L'utente dispone di un username univoco e un id identificativo.
- Le credenziali per l'accesso sono la coppia username e password.
- L'utente loggato nell'applicazione può **effettuare il logout** o **ritornare alla home page** da tutte le pagine dell'applicazione.
- Esiste una **pagina per la REGISTRAZIONE** di nuovi utenti raggiungibile dalla pagina di login.
- Se l'utente inserisce dati sbagliati verrà se possibile **reindirizzato** alla pagine di provenienza o **reindirizzato** a una **pagina di ERRORE** che specifica la causa.
- Dopo che un'asta è scaduta non è più possibile offrire anche se non è ancora stata chiusa dall'utente che l'ha creata
- Le aste senza offerte che vengono chiuse reinseriscono gli articoli come disponibili.

Components:

Model objects (beans):

- User
- Auction
- Item
- Bid

Controllers (servlets):

- Login
- Register
- GoToLogin
- GoToRegister
- GoToSell
- GoToBuy
- CreateAuction
- CreateItem
- CloseAuction
- GoToAuctionDetails
- GoToAuctionBids
- RegisterBid
- SearchByKeyword
- Logout

Data Access Objects (DAO) (first part):

- UserDao:
 - findUser(username,password) : user
 - registerUser(username, password, address) : void
 - getUserById(id_user) : user
 - getUserByUsername(username) : user
- AuctionDAO
 - getOpenedAuctionsByUserId(id_user) : list<Auction>
 - getClosedAuctionsByUserId(id_user) : list<Auction>
 - createAuction(id_user, bid_increment, closing_date, ids_item) : void
 - getAuctionDetailsByAuctionId(id_auction) : Auction
 - isMyAuction(id_user, id_auction) : boolean
 - closeAuction(id_auction) : void
 - searchByKeyword(keyword) : list<Auction>
 - getBidIncrementByAuctionId(id_auction) : int
 - getStartingBidByAuctionId(id_auction) : int
 - isClosed(id_auction) : boolean
 - canBeClosed(id_auction) : boolean

Components:

Data Access Objects (DAO) (second part):

- ItemDAO:
 - getItemById(id_item) : item
 - allItemMine(id_user, ids_item) : boolean
 - publishItem(id_user, price, name, description, img_name) : void
 - getNotAuctionedItemById(id_item) : list<Item>
 - getWonItemsByAuctionIds(ids_auction) : list<Item>
 - getItemsByAuctionId(id_auction) : list<Item>
- BidDAO:
 - getBidsByAuctionId(id_auction) : list<Bid>
 - registerBid(id_user, id_auction, price) : void
 - getBuyerIdByAuctionId(id_auction) : int
 - getAwardedAuctionsByUserId(id_user) : list<Bid>
 - getMaxBidByAuctionId(id_auction) : int

Views (Templates):

login.html
register.html
home.html
buy.html
sell.html
auction_bids.html
auction_details.html
error.html
header.html

Filters:

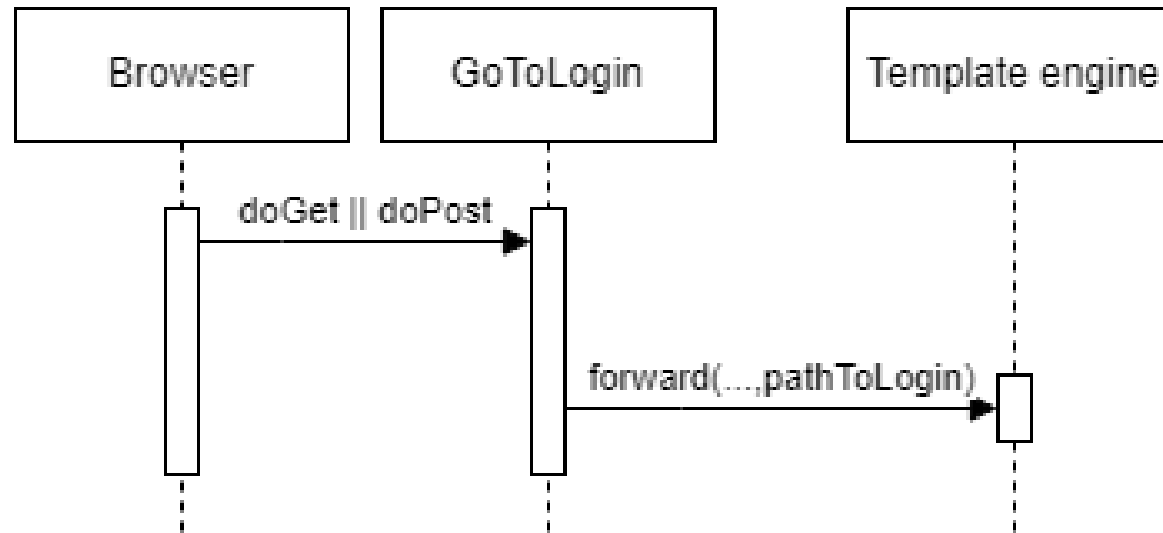
LoggedInUser
NotLoggedInUser

Sequence Diagram

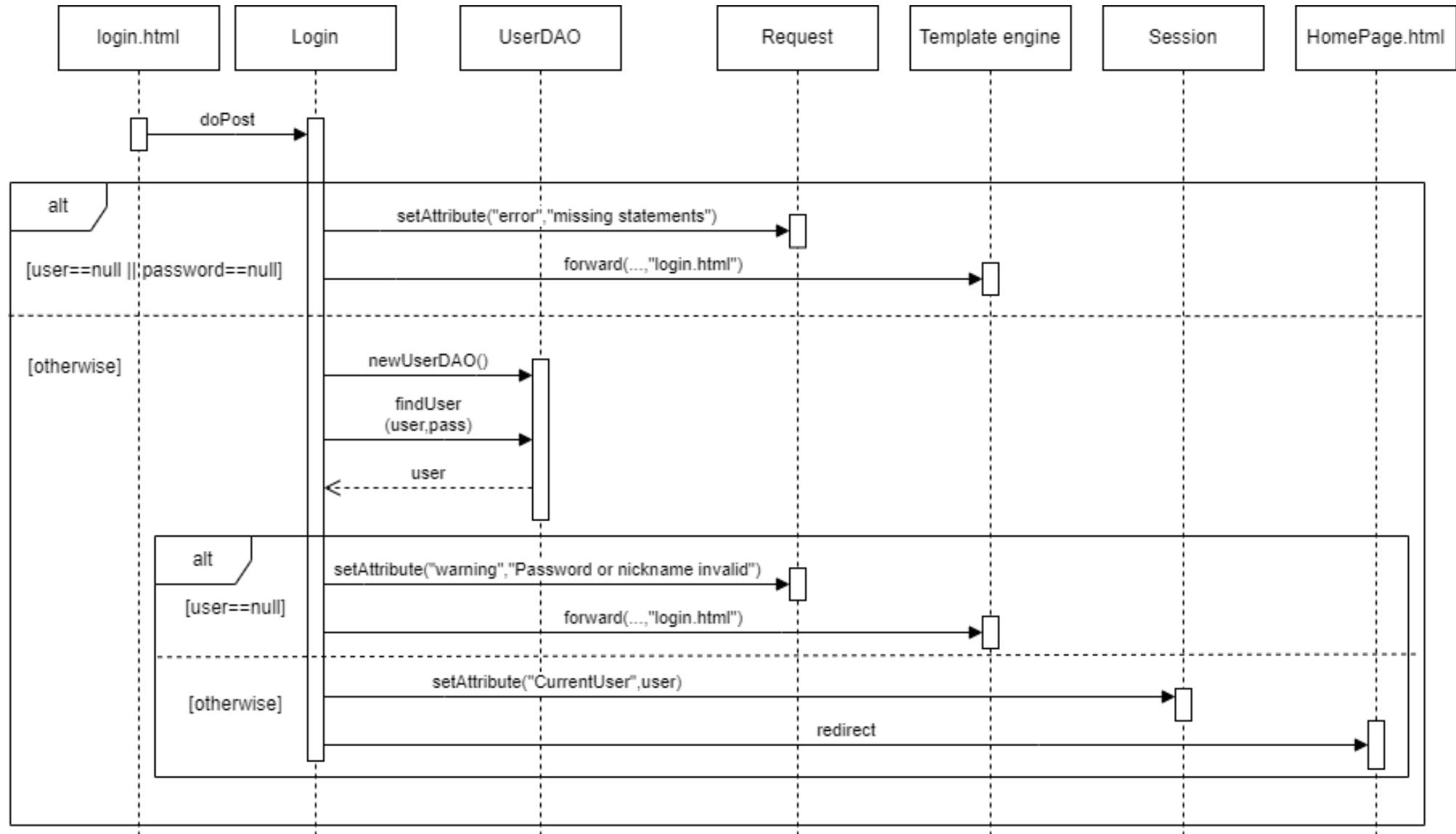
Per semplicità sono stati omessi i controlli sui parametri in ingresso.

Ogni richiesta al server passa attraverso uno dei due filtri (LoggedUser se l'utente ha già fatto il login, altrimenti NotLoggedUser), che verificano se la sessione è valida e se l'utente è loggato.

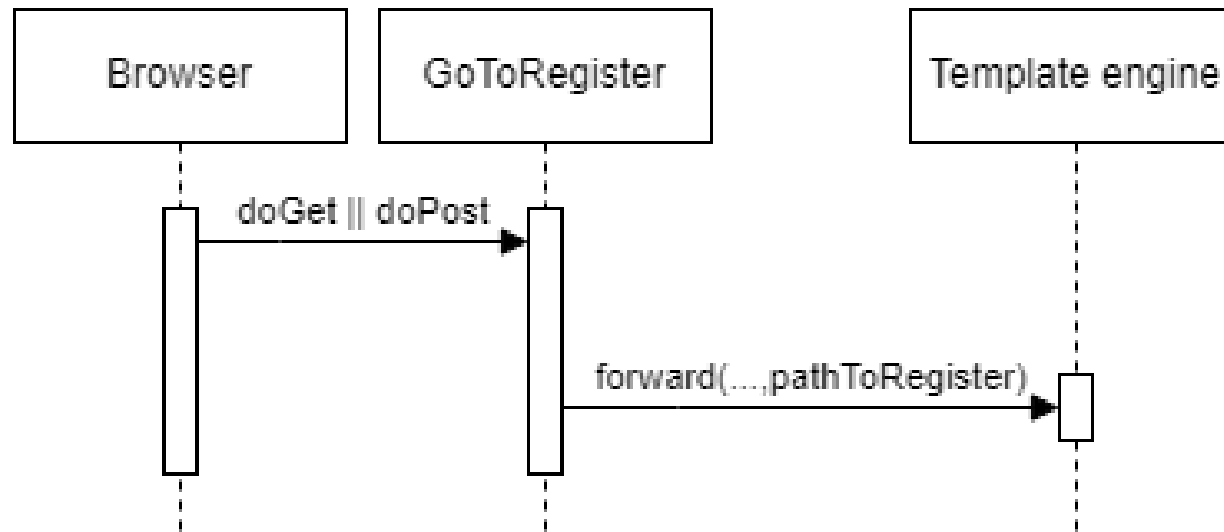
GoToLogin



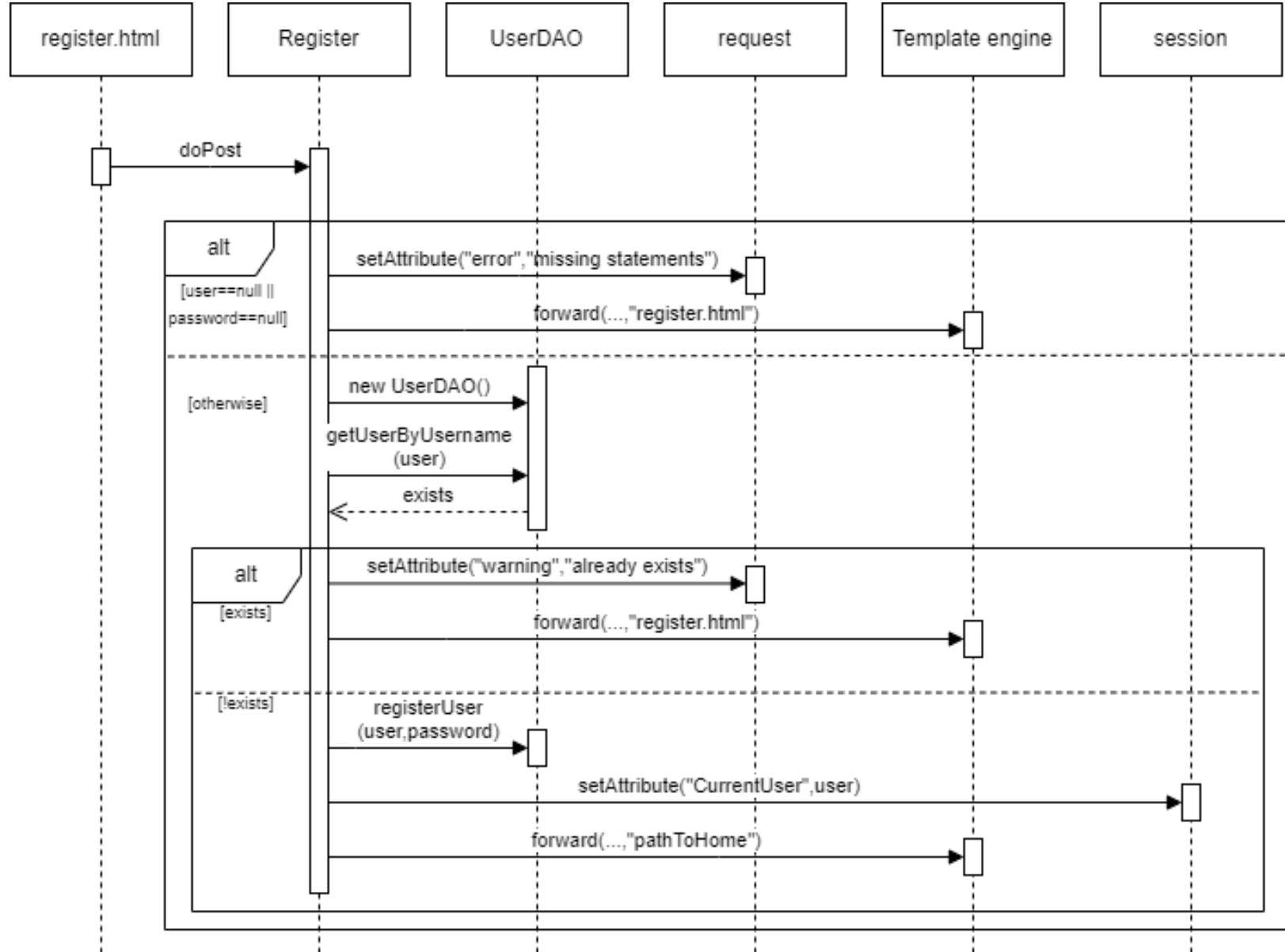
Login



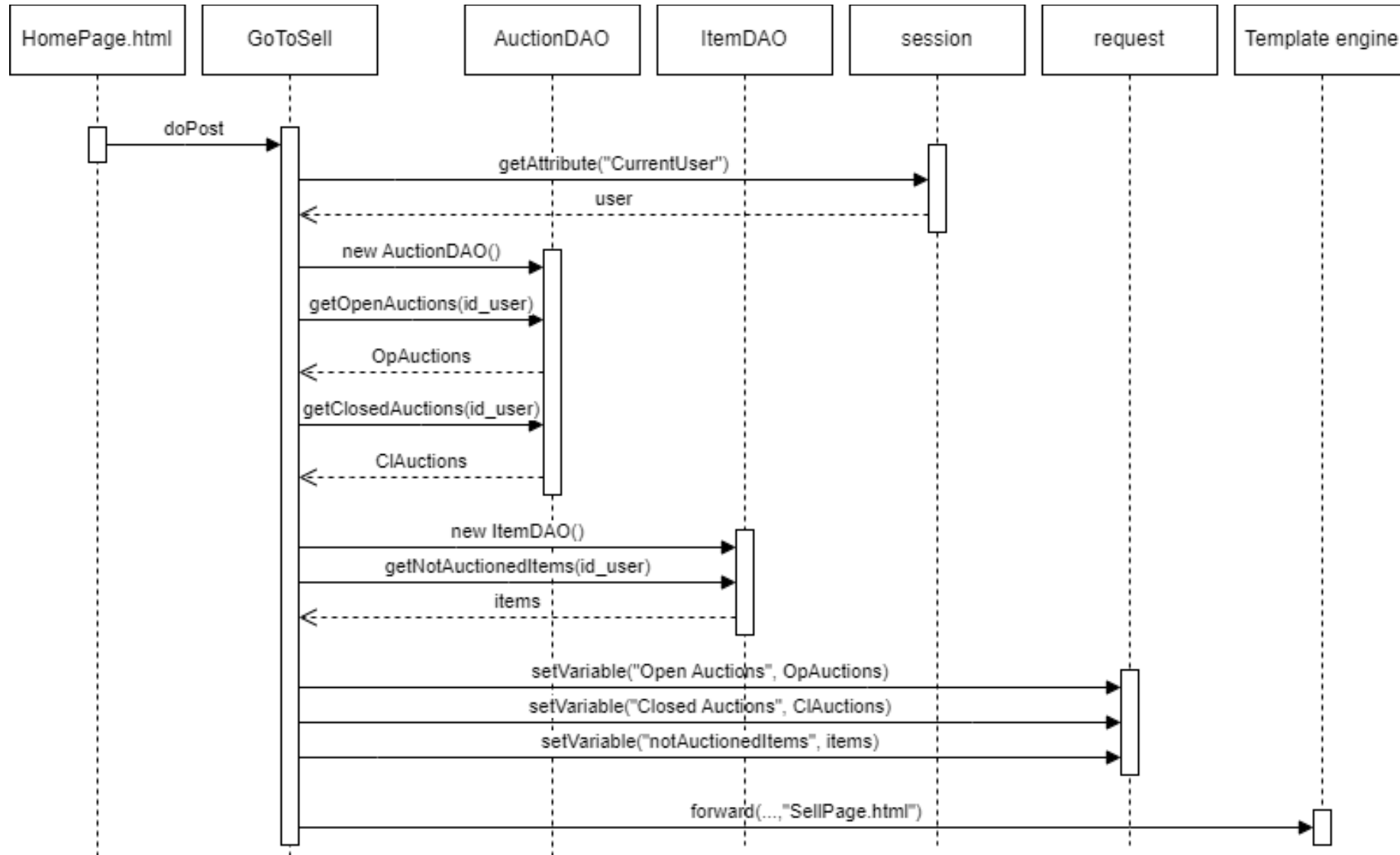
GoToRegister



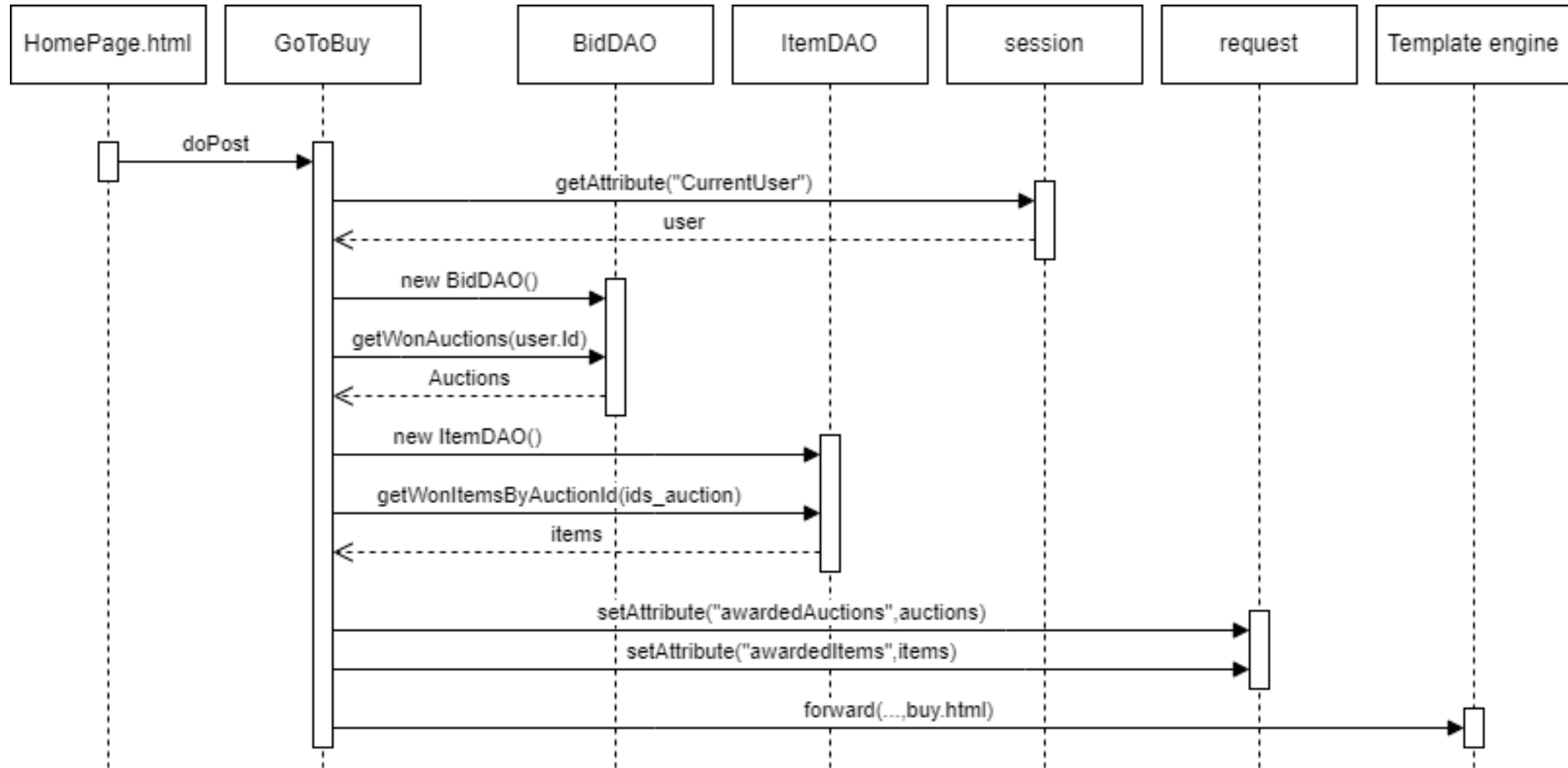
Register



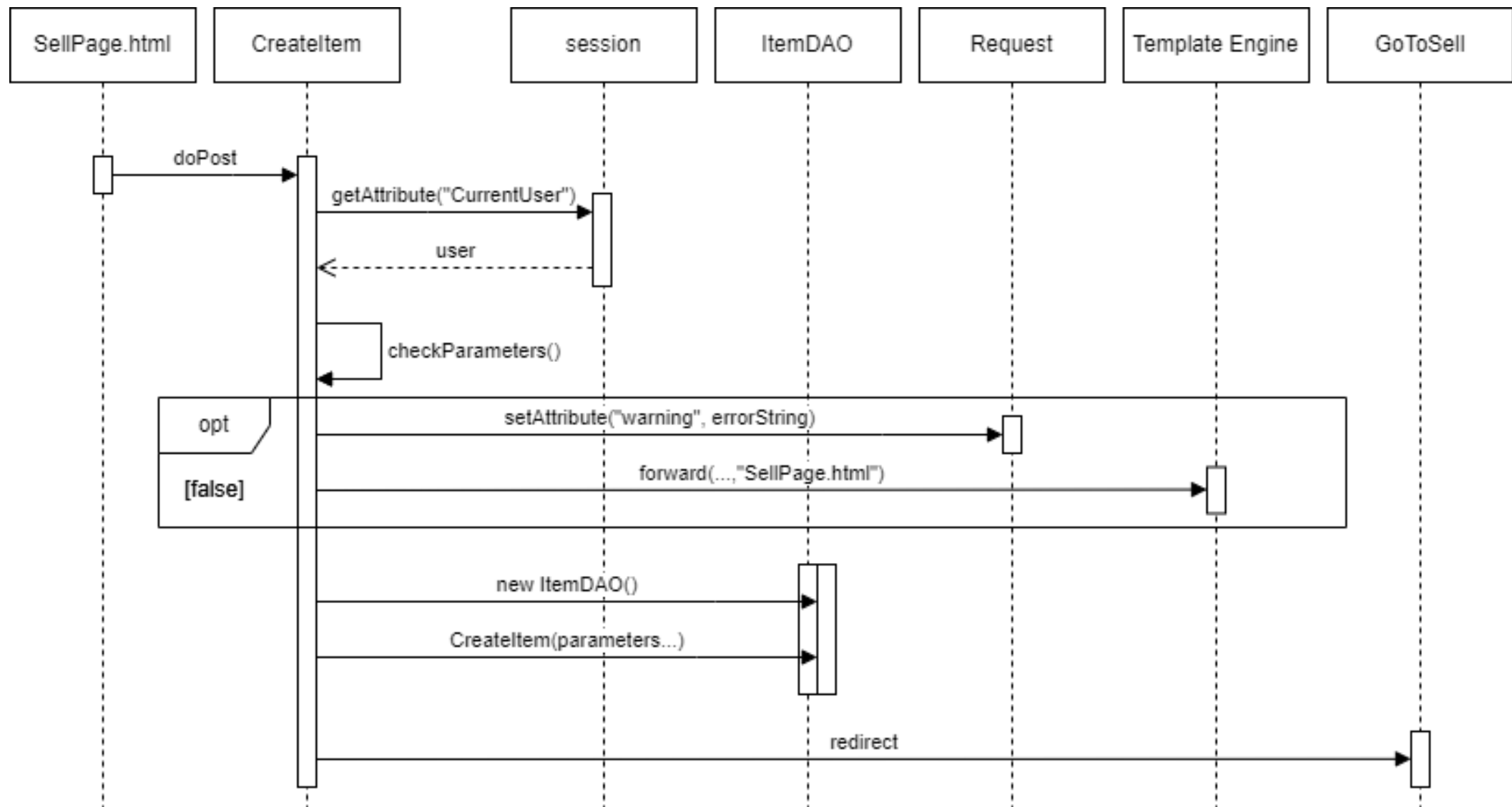
GoToSell



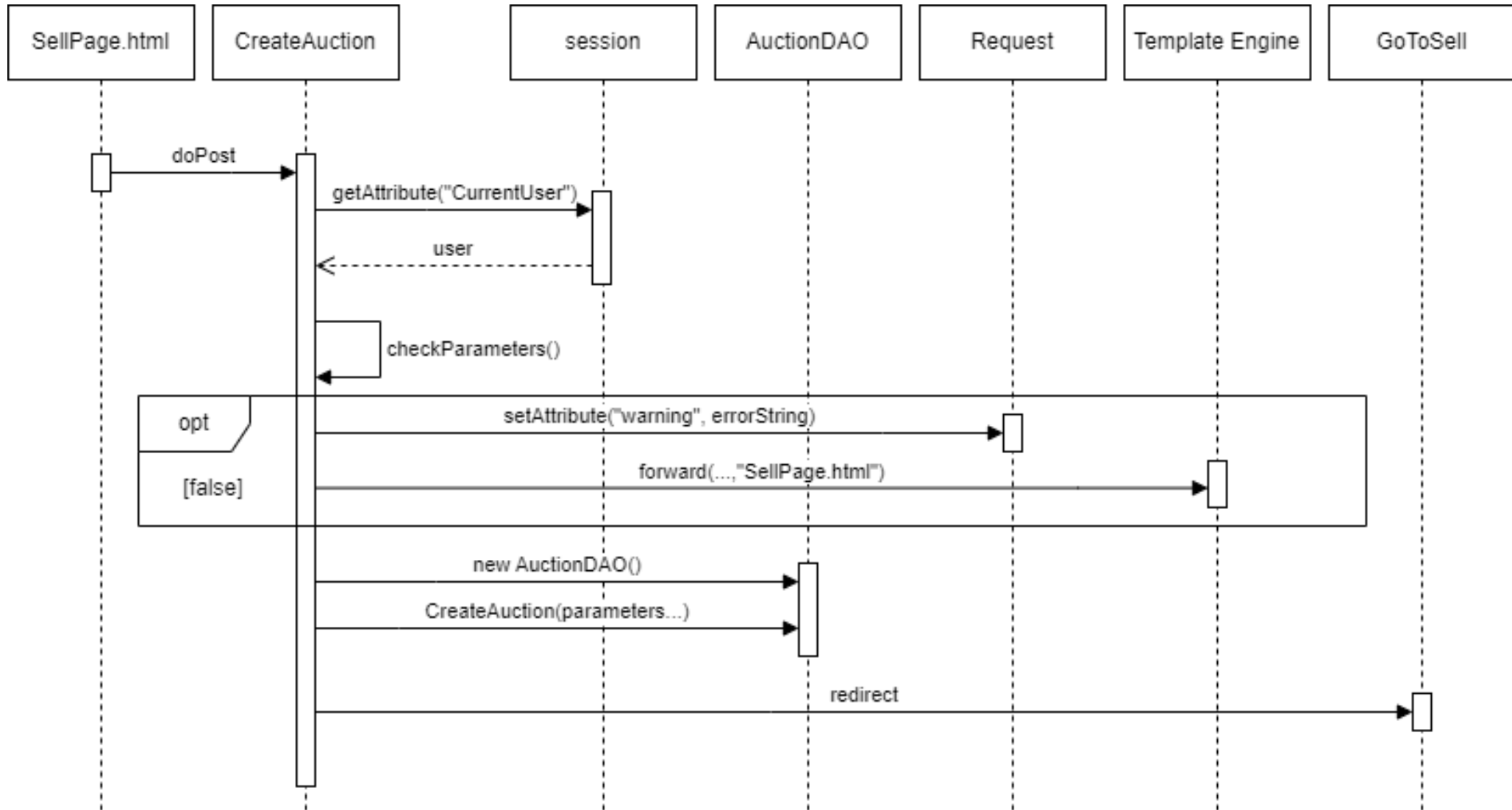
GoToBuy



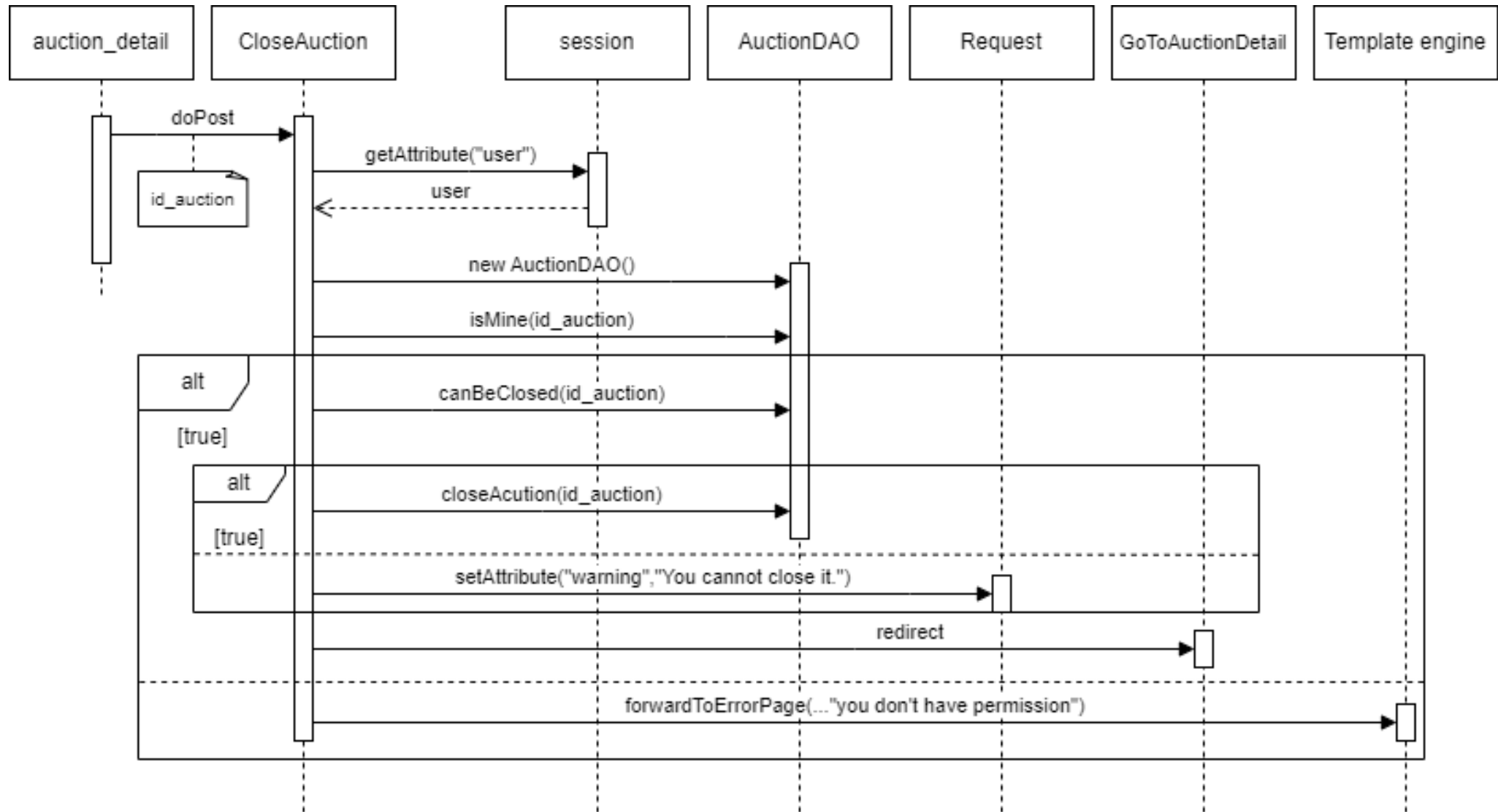
CreateItem



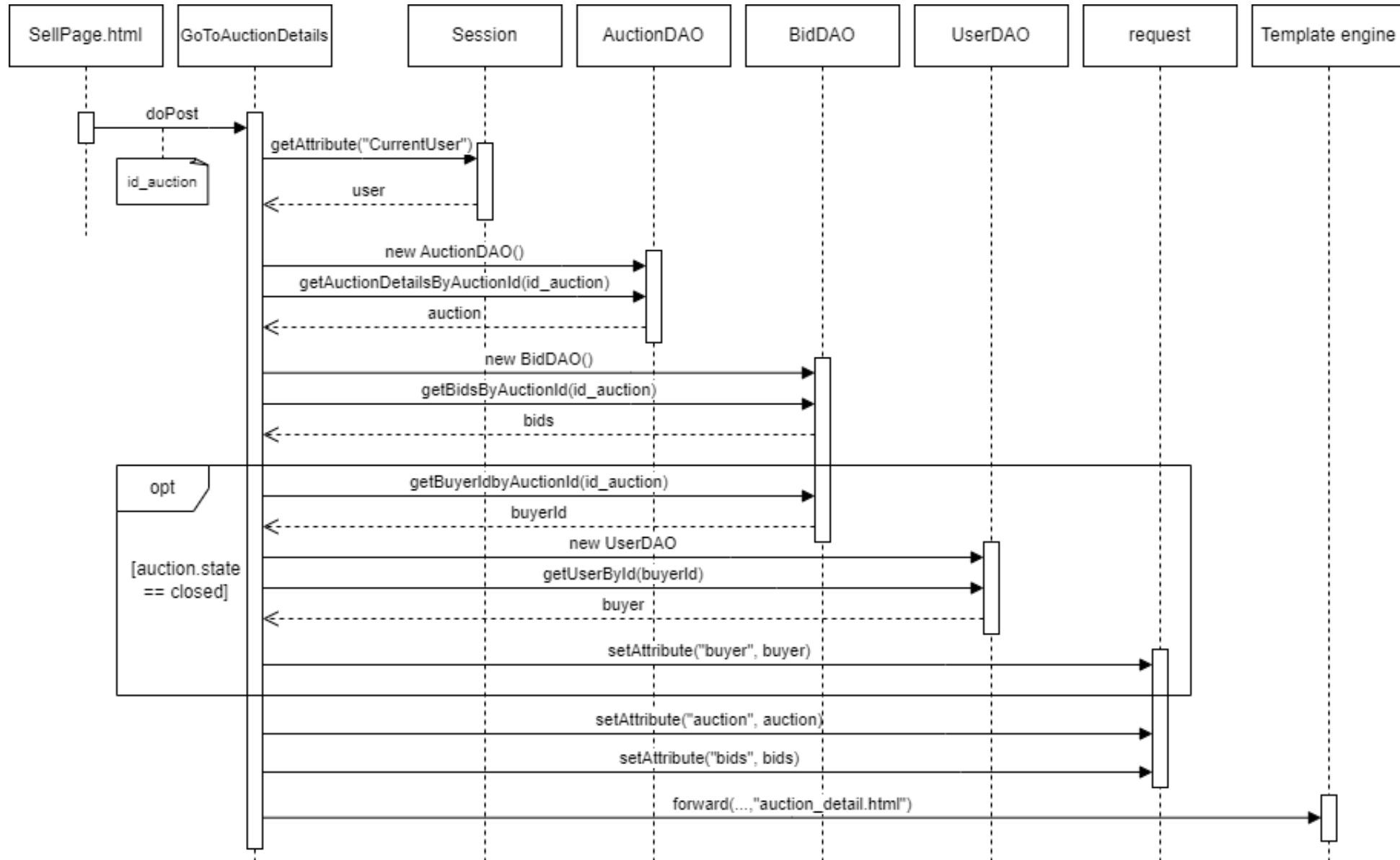
CreateAuction



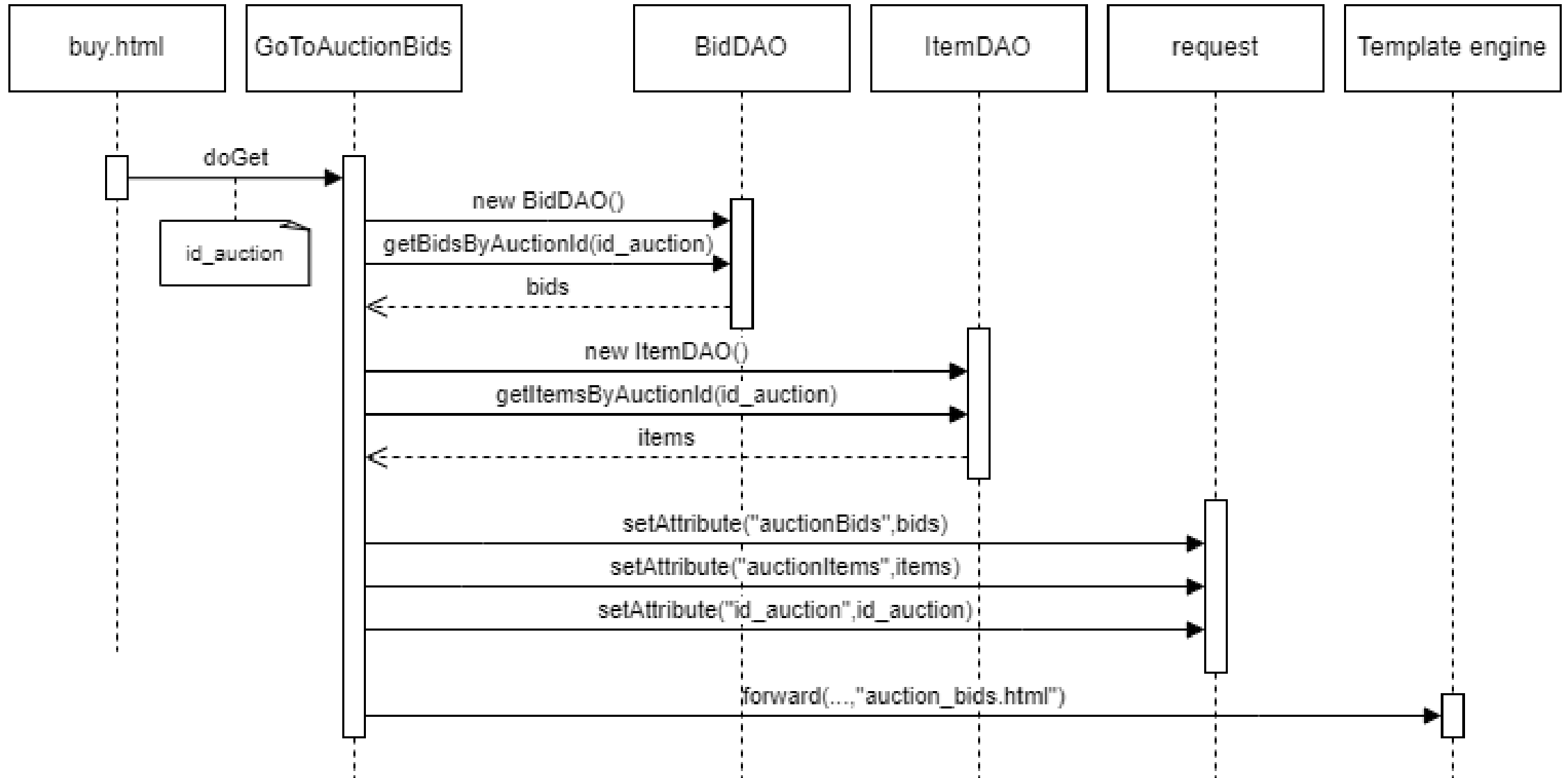
CloseAuction



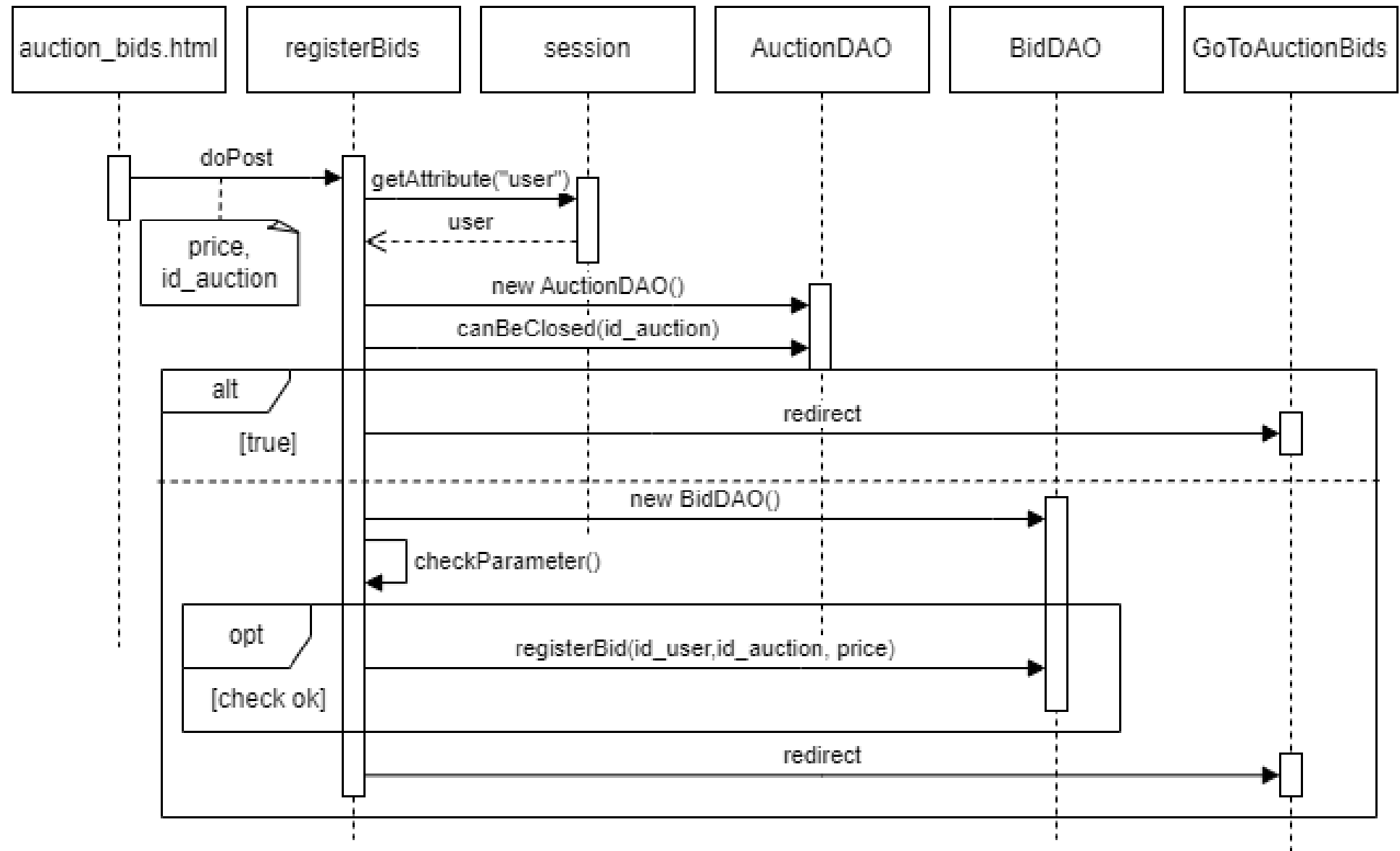
GoToAuctionDetails



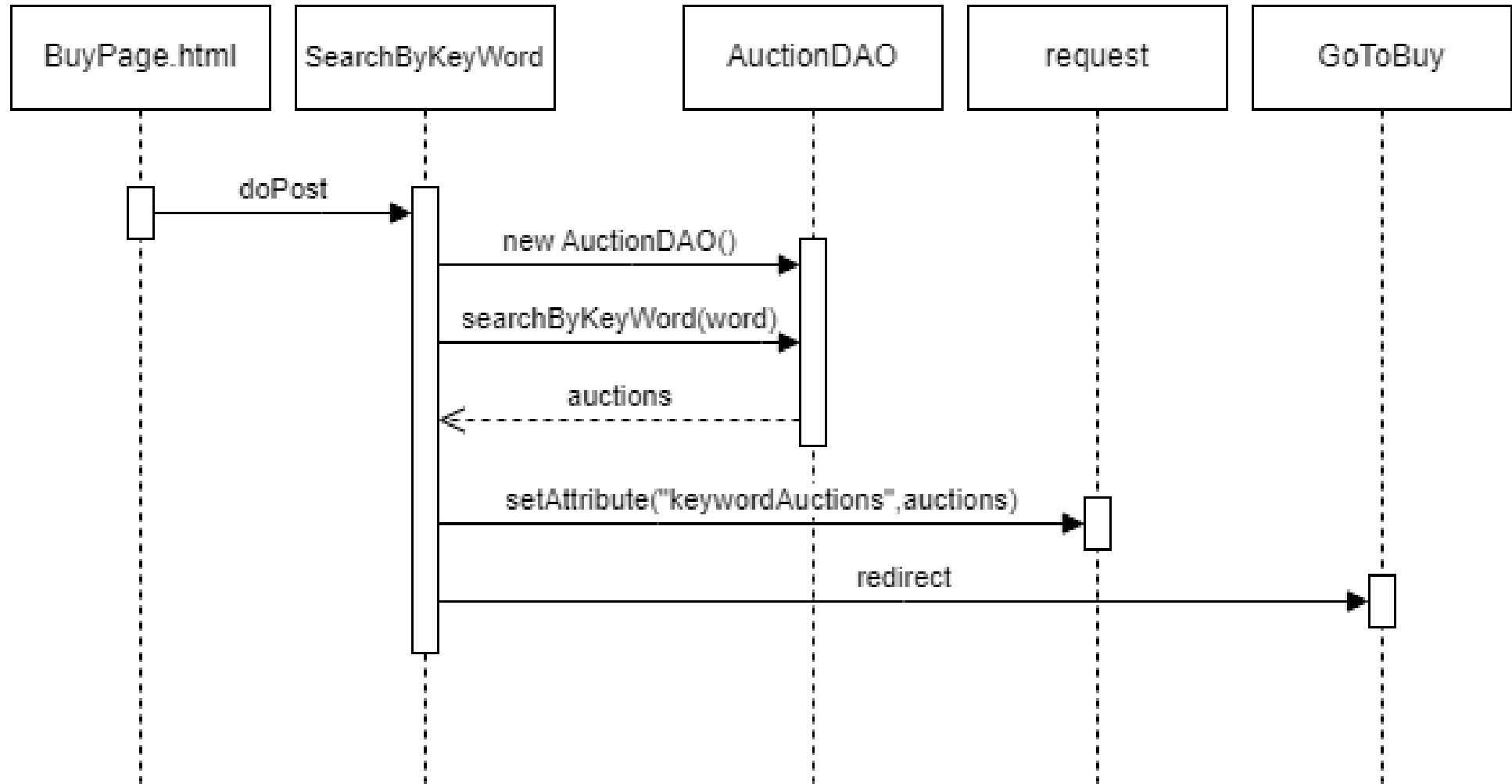
GoToAuctionBids



RegisterBid



SearchByKeyword



Logout

